

Dr. H N National College of Engineering

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi and Govt. Of Karnataka)

36B Cross, Jayanagar 7th block, Bengaluru – 560070



Third Year
A Mini-Project Report
On
“Smart Retail Billing System”

Submitted in the partial fulfilment of the requirements for

ADVANCED JAVA

(BIS402)

SUBMITTED BY :

Rishitha R S (1HC24IS038)

Rohith M K (1HC24IS039)

Sagar G R (1HC24IS043)

Srilakshmi R (1HC24IS051)

IVth SEMESTER

Under the Guidance of
Dr. Nandita

DEPT. OF INFORMATION SCIENCE AND ENGINEERING

Dr. H N National College of Engineering

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi
and Govt. of Karnataka)

Department of ISE



2025-2026

Certificate

This is to certify that the Mini Project Work entitled “**Smart Retail Billing System**” is a bonafide work carried out by **Rishitha R S (1HC24IS038)**, **Rohith M K (1HC24IS039)**, **Sagar G R (1HC24IS043)** and **Srilakshmi R (1HC24IS051)**, in partial fulfilment for the IV semester of Bachelor of Engineering in ISE of the Visvesvaraya Technological University, Belgaum during the year 2025-2026. The Project report has been approved as it satisfies the academics prescribed for the Bachelor of Engineering degree.

Signature of Guide

Signature of HOD

Contents:	Page
1. INTRODUCTION	1
2. PROBLEM DESCRIPTION	2
3. HIGH LEVEL DESIGN	3
4. DETAIL LEVEL DESIGN	4
5. CODE	5
6. RESULT	25
7. PROGRAM EXPLANATION	32
8. CONCLUSION	33
9. HARDWARE & SOFTWARE REQUIREMENTS	33
10. REFERENCES	39

CHAPTER 1

INTRODUCTION

Java is one of the most widely used object-oriented programming languages because of its platform independence, robustness, and security. Object-Oriented Programming (OOP) provides a structured and modular approach to software development using classes and objects.

The Smart Retail Billing System is a web-based application developed to simplify retail shop billing and inventory management. Manual billing systems are time-consuming and prone to errors in calculations and stock maintenance.

This project is developed using Java Servlets, HTML, CSS, JavaScript, JDBC, Apache Tomcat Server and MySQL database connectivity.

The system allows users to:

- Login and signup

- Add products

- Manage stock

- Add products to cart

- Generate bills

- Automatically update stock after billing

The project demonstrates the practical implementation of Java web technologies and object-oriented programming concepts

CHAPTER 2

PROBLEM DESCRIPTION

In many small retail shops, billing and stock management are still performed manually. This causes:

- Billing errors
- Incorrect stock calculations
- Increased billing time
- Difficulty maintaining product records
- Poor inventory tracking

There is a need for a computerized billing system that can automate product management and billing operations efficiently.

The Smart Retail Billing System solves these issues by providing:

- Digital product storage
- Fast billing system
- Shopping cart management
- Automatic stock updates
- Accurate total calculations

CHAPTER 3

HIGH LEVEL DESIGN

The High Level Design (HLD) describes the overall architecture of the Smart Retail Billing System.

System Modules

1. Frontend Layer

Developed using:

HTML

CSS

JavaScript

Functions:

Login and Signup Interface

Product Management

Cart Display

Bill Generation

2. Backend Layer

Developed using Java Servlets.

Functions:

Handle product insertion

Process stock updates

Connect frontend with database

Servlets Used:

ProductServlet

UpdateStockServlet

HomeServlet

3. Database Layer

Developed using MySQL.

Stores:

Product Name

Product Price

Product Quantity

System Workflow

User logs into the system.

Products are added through frontend forms.

Product details are stored in MySQL database.

Products are displayed on screen.

User adds products into cart.

Bill is generated dynamically.

Database stock quantity is reduced automatically.

CHAPTER 4

DETAILED LEVEL DESIGN

DBConnection Class

Used to establish connection between Java application and MySQL database using JDBC.

Responsibilities

Load JDBC Driver

Create database connection

Return connection object

ProductServlet Class

Handles insertion of products into database.

Responsibilities

Receive product details from frontend

Execute INSERT SQL query

Display success/failure message

UpdateStockServlet Class

Updates stock quantity after billing.

Responsibilities

Receive purchased quantity

Reduce stock in database

Execute UPDATE SQL query

HomeServlet Class

Redirects application to index.html page.

OBJECT ORIENTED PROGRAMMING CONCEPTS USED

1. Class and Object

Classes are used to organize program logic.

DBConnection

ProductServlet

UpdateStockServlet

2. Encapsulation

Database operations are enclosed inside Java classes and methods.

3. Abstraction

Complex backend and database logic are hidden from users.

4. Modularity

The system is divided into independent modules:

Frontend

Backend

Database Layer

5. JDBC Connectivity

Used to establish communication between Java application and MySQL database. Ensures smooth program execution.

CHAPTER 5

CODE (Frontend Code)

```
<!DOCTYPE html>
<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Smart Retail Billing</title>

<style>

*{
    margin:0;
    padding:0;
    box-sizing:border-box;
    font-family:Arial,sans-serif;
}

body{
    background:linear-gradient(135deg,#1e3c72,#2a5298);
    min-height:100vh;
    color:white;
    text-align:center;
    padding:20px;
}

h1{
    margin-bottom:20px;
    font-size:45px;
}

h2{
    margin:20px 0;
}

.card{
    width:400px;
    margin:20px auto;
    background:rgba(255,255,255,0.1);
    padding:25px;
    border-radius:15px;
    backdrop-filter:blur(10px);
}
```


Smart Retail Billing System

```
input{
    width:100%;
    padding:12px;
    margin:10px 0;
    border:none;
    border-radius:8px;
    font-size:16px;
}

button{
    padding:10px 20px;
    border:none;
    border-radius:8px;
    cursor:pointer;
    margin:5px;
    font-weight:bold;
}

.primary{
    background:#32e0c4;
    color:black;
}

.delete{
    background:#ff4d4d;
    color:white;
}

.green{
    background:#28a745;
    color:white;
}

.hidden{
    display:none;
}

table{
    width:95%;
    margin:20px auto;
    border-collapse:collapse;
    background:white;
    color:black;
}

th{
    background:#0d1117;
    color:white;
}

th,td{
    padding:15px;
    border:1px solid #ddd;
}
```

```
.totalBox{
    font-size:35px;
    margin-top:20px;
    font-weight:bold;
}

.invoiceBtn{
    margin-top:15px;
}

.logoutBtn{
    background:white;
}

</style>

</head>

<body>

<h1>Smart Retail Billing</h1>

<!-- LOGIN PAGE -->

<div id="loginPage" class="card">

    <h2>Login</h2>

    <input type="text"
        id="loginUser"
        placeholder="Username">

    <input type="password"
        id="loginPass"
        placeholder="Password">

    <button class="primary"
        onclick="login()">
        Login
    </button>

    <br>

    <button onclick="showSignup()">
        Create Account
    </button>

</div>

<!-- SIGNUP PAGE -->

<div id="signupPage" class="card hidden">
```

Smart Retail Billing System

```
<h2>Create Account</h2>

<input type="text"
      id="signupUser"
      placeholder="Username">

<input type="password"
      id="signupPass"
      placeholder="Password">

<button class="primary"
      onclick="signup()">
  Sign Up
</button>

<br>

<button onclick="showLogin()">
  Back to Login
</button>

</div>

<!-- MAIN APPLICATION -->

<div id="appPage" class="hidden">

  <h2>Billing Dashboard</h2>

  <button class="logoutBtn"
    onclick="logout()">
    Logout
  </button>

  <!-- CUSTOMER DETAILS -->

  <div class="card">

    <h2>Customer Details</h2>

    <input type="text"
      id="customerName"
      placeholder="Customer Name">

  </div>

  <!-- ADD PRODUCT -->

  <div class="card">

    <h2>Add Product</h2>

    <input type="text"
      id="name">
```

Smart Retail Billing System

```
        placeholder="Product Name">

        <input type="number"
            id="price"
            placeholder="Price">

        <input type="number"
            id="stock"
            placeholder="Stock">

        <button class="primary"
            onclick="addProduct()">
            Add Product
        </button>

</div>

<!-- PRODUCT TABLE -->

<h2>Products</h2>

<table>

    <thead>

        <tr>

            <th>Name</th>
            <th>Price</th>
            <th>Stock</th>
            <th>Qty</th>
            <th>Action</th>

        </tr>

    </thead>

    <tbody id="productTable"></tbody>

</table>

<!-- CART TABLE -->

<h2>Cart</h2>

<table>

    <thead>

        <tr>

            <th>Name</th>
            <th>Qty</th>
            <th>Price</th>
```

Smart Retail Billing System

```
                <th>Total</th>
                <th>Action</th>

            </tr>

        </thead>

        <tbody id="cartTable"></tbody>

    </table>

    <!-- GRAND TOTAL -->

    <div class="totalBox"
        id="grandTotal">

        Total: ₹0

    </div>

    <!-- BILL BUTTONS -->

    <div class="invoiceBtn">

        <button class="primary"
            onclick="generateBill()">
            Generate Bill
        </button>

        <button onclick="clearCart()">
            Clear Cart
        </button>

    </div>

</div>

<script>

let products = [];

let cart = [];

let grandTotal = 0;

/* =====
   SHOW SIGNUP PAGE
===== */

function showSignup(){

    document.getElementById("loginPage")
        .classList.add("hidden");
```

Smart Retail Billing System

```
        document.getElementById("signupPage")
            .classList.remove("hidden");
    }

    /* =====
    SHOW LOGIN PAGE
    ===== */

    function showLogin(){

        document.getElementById("signupPage")
            .classList.add("hidden");

        document.getElementById("loginPage")
            .classList.remove("hidden");
    }

    /* =====
    SIGNUP FUNCTION
    ===== */

    function signup(){

        let user =
            document.getElementById("signupUser").value;

        let pass =
            document.getElementById("signupPass").value;

        if(user === "" || pass === ""){

            alert("Enter details");
            return;
        }

        localStorage.setItem("user", user);
        localStorage.setItem("pass", pass);

        alert("Account Created");

        showLogin();
    }

    /* =====
    LOGIN FUNCTION
    ===== */

    function login(){

        let user =
            document.getElementById("loginUser").value;

        let pass =
            document.getElementById("loginPass").value;
```

```
let savedUser =
    localStorage.getItem("user");

let savedPass =
    localStorage.getItem("pass");

if(user === savedUser &&
    pass === savedPass){

    document.getElementById("loginPage")
        .classList.add("hidden");

    document.getElementById("signupPage")
        .classList.add("hidden");

    document.getElementById("appPage")
        .classList.remove("hidden");

} else {

    alert("Invalid Login");
}

}

/* =====
  LOGOUT FUNCTION
===== */

function logout(){

    location.reload();
}

/* =====
  ADD PRODUCT
===== */

function addProduct(){

    let name =
        document.getElementById("name").value;

    let price =
        document.getElementById("price").value;

    let stock =
        document.getElementById("stock").value;

    if(name === "" ||
        price === "" ||
        stock === ""){

        alert("Enter all fields");
```

```
        return;
    }

    fetch("/SmartRetailServlet/addProduct", {

        method: "POST",

        headers: {
            "Content-Type":
                "application/x-www-form-urlencoded"
        },

        body:
            "name=" + encodeURIComponent(name)
            + "&price=" + encodeURIComponent(price)
            + "&quantity=" + encodeURIComponent(stock)

    })

    .then(response => response.text())

    .then(data => {

        alert(data);

        let product = {

            name: name,
            price: parseFloat(price),
            stock: parseInt(stock)

        };

        products.push(product);

        displayProducts();

        document.getElementById("name").value = "";
        document.getElementById("price").value = "";
        document.getElementById("stock").value = "";

    })

    .catch(error => {

        console.log(error);

        alert("Error");

    });
}

/* =====
   DISPLAY PRODUCTS
   ===== */
```



```
function displayProducts(){

    let table =
        document.getElementById("productTable");

    table.innerHTML = "";

    products.forEach((product,index)=>{

        table.innerHTML += `

        <tr>

            <td>${product.name}</td>
            <td>${product.price}</td>
            <td>${product.stock}</td>
            <td>1</td>

            <td>

                <button class="green"
                    onclick="addToCart(${index})">

                    Add

                </button>

            </td>

        </tr>

        `;

    });
}

/* =====
   ADD TO CART
   ===== */

function addToCart(index){

    let product = products[index];

    if(product.stock <= 0){

        alert("Out of Stock");
        return;
    }

    let existing =
        cart.find(item =>
            item.name === product.name);
```

```
        if(existing){

            existing.qty += 1;

            existing.total =
                existing.qty * existing.price;

        } else {

            cart.push({

                name: product.name,
                qty: 1,
                price: product.price,
                total: product.price

            });

        }

        product.stock -= 1;

        displayProducts();

        displayCart();
    }

    /* =====
    DISPLAY CART
    ===== */

    function displayCart(){

        let table =
            document.getElementById("cartTable");

        table.innerHTML = "";

        grandTotal = 0;

        cart.forEach((item,index)=>{

            grandTotal += item.total;

            table.innerHTML += `

                <tr>

                    <td>${item.name}</td>
                    <td>${item.qty}</td>
                    <td>${item.price}</td>
                    <td>${item.total}</td>

                <td>
```

Smart Retail Billing System

```
                <button class="delete"
                    onclick="deleteCart(${index})">

                    Delete

                </button>

            </td>

        </tr>

    `;
    });

    document.getElementById("grandTotal")
        .innerHTML =
        "Total: ₹" + grandTotal;
}

/* =====
DELETE CART ITEM
===== */

function deleteCart(index){

    cart.splice(index,1);

    displayCart();
}

/* =====
CLEAR CART
===== */

function clearCart(){

    cart = [];

    displayCart();
}

/* =====
GENERATE BILL
===== */

function generateBill(){

    if(cart.length === 0){

        alert("Cart Empty");
        return;
    }

    cart.forEach(item => {
```

```
        fetch("/SmartRetailServlet/updateStock", {
            method: "POST",
            headers: {
                "Content-Type":
                    "application/x-www-form-urlencoded"
            },
            body:
                "name=" + encodeURIComponent(item.name)
                + "&qty=" + encodeURIComponent(item.qty)
        });
    });

    let customer =
        document.getElementById("customerName").value;

    let bill = `
SMART RETAIL BILL

Customer: ${customer}

`;

    cart.forEach(item => {

        bill += `

${item.name}
Qty: ${item.qty}
Price: ₹${item.price}
Total: ₹${item.total}

`;

    });

    bill += `

Grand Total: ₹${grandTotal}

`;

    let newWindow =
        window.open("", "_blank");

    newWindow.document.write(`

<html>
```

```
<head>

<title>Bill</title>

<style>

body{
    font-family:Arial;
    padding:20px;
}

h1{
    text-align:center;
}

pre{
    font-size:18px;
}

</style>

</head>

<body>

<h1>SMART RETAIL BILL</h1>

<pre>${bill}</pre>

</body>

</html>

`);

newWindow.print();
}

</script>

</body>

</html>
```

CHAPTER 5

CODE (Backend Code)

1. HomeServlet.java

```
package com.smartretail;

import java.io.IOException;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/home")

public class HomeServlet extends HttpServlet {

    @Override

    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)

        throws ServletException, IOException {

        response.sendRedirect("index.html");
    }
}
```

2. DBConnection.java

```
package com.smartretail;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    public static Connection getConnection() {
```

```
        Connection con = null;

        try {

            // Load MySQL Driver

            Class.forName("com.mysql.cj.jdbc.Driver");

            // Create Database Connection

            con = DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/smartretail",

                "root",

                "root@2005"

            );

            System.out.println("Database Connected");

        } catch (Exception e) {

            e.printStackTrace();

        }

        return con;

    }

}
```

3. ProductServlet.java

```
package com.smartretail;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.PreparedStatement;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/addProduct")

public class ProductServlet extends HttpServlet {

    protected void doPost(
        HttpServletRequest request,
        HttpServletResponse response)

        throws ServletException, IOException {

        response.setContentType("text/plain");

        PrintWriter out = response.getWriter();

        try {

            // Get Product Details

            String name =
                request.getParameter("name");

            double price =
                Double.parseDouble(
                    request.getParameter("price"));

            int quantity =
                Integer.parseInt(
                    request.getParameter("quantity"));

            // Database Connection

            Connection con =
                DBConnection.getConnection();

            // SQL Query

            String query =
                "INSERT INTO products(name, price, quantity)
VALUES(?, ?, ?)";
```



```
        PreparedStatement ps =
            con.prepareStatement(query);

        // Set Values

        ps.setString(1, name);

        ps.setDouble(2, price);

        ps.setInt(3, quantity);

        // Execute Query

        int rows = ps.executeUpdate();

        // Check Result

        if(rows > 0) {

            out.print("Product Added Successfully");

        } else {

            out.print("Insert Failed");

        }

        // Close Connection

        ps.close();

        con.close();

    } catch(Exception e) {

        e.printStackTrace();

        out.print("Error : " + e.getMessage());

    }

}
```

4. UpdateStockServlet.java

Java

```
package com.smartretail;

import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/updateStock")

public class UpdateStockServlet extends HttpServlet {

    protected void doPost(
        HttpServletRequest request,
        HttpServletResponse response)

        throws ServletException, IOException {

        response.setContentType("text/plain");

        try {

            // Get Product Name

            String name =
                request.getParameter("name");

            // Get Quantity

            int qty =
                Integer.parseInt(
                    request.getParameter("qty"));

            // Database Connection
```

```
        Connection con =
            DBConnection.getConnection();

        // SQL Query
        String query =
            "UPDATE products SET quantity = quantity - ? WHERE name = ?";
        PreparedStatement ps =
            con.prepareStatement(query);

        // Set Values
        ps.setInt(1, qty);

        ps.setString(2, name);

        // Execute Query
        int rows = ps.executeUpdate();

        // Check Result

        if(rows > 0) {

            response.getWriter()
                .print("Stock Updated");
        } else {
            response.getWriter()
                .print("Product Not Found");
        }

        // Close Connection
        ps.close();
        con.close();

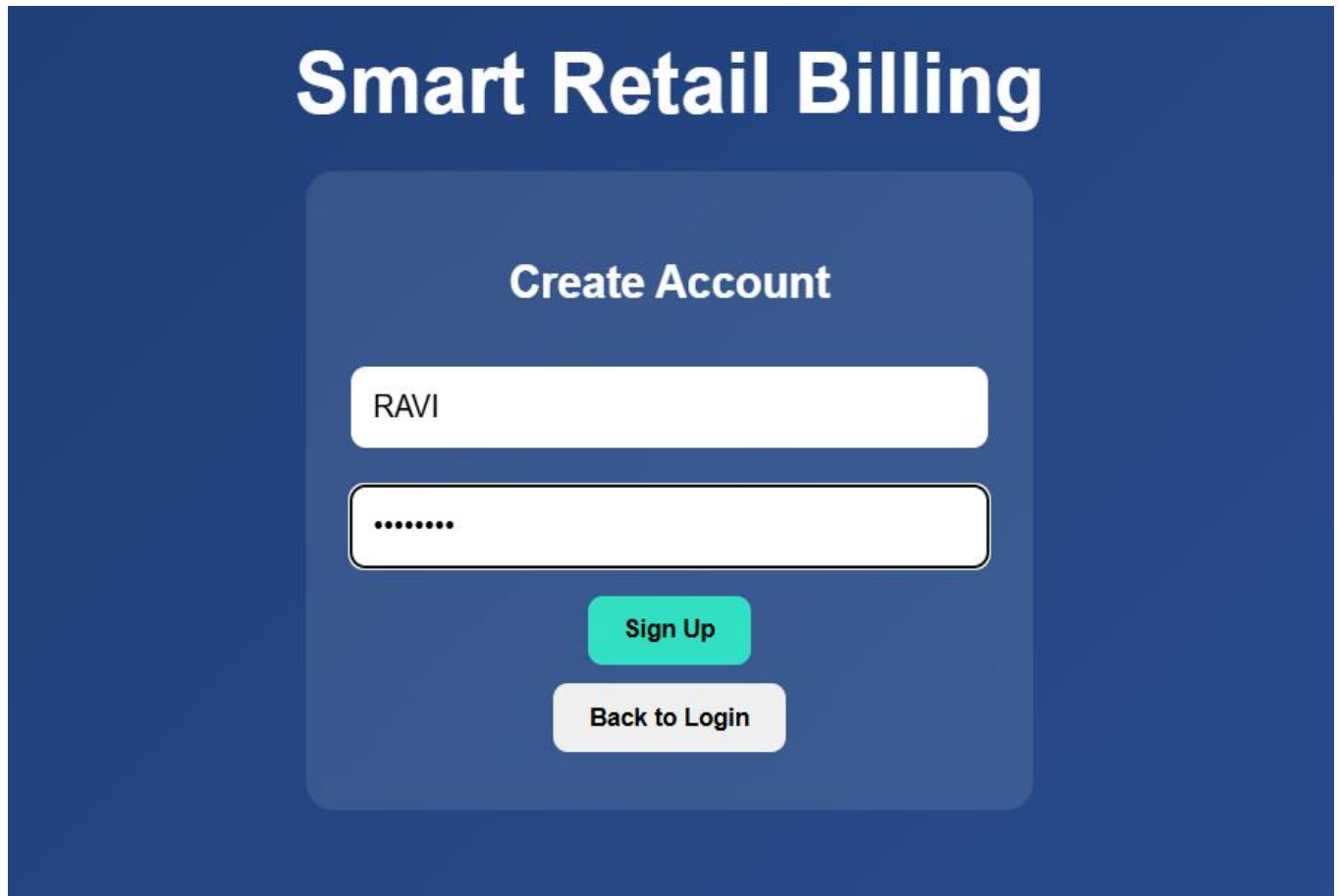
    } catch(Exception e) {

        e.printStackTrace();

        response.getWriter()
            .print("Error : " + e.getMessage());
    }
}
```

CHAPTER 6

OUTPUT (Frontend Output)



The image shows a web form titled "Smart Retail Billing" with a "Create Account" section. The form has a dark blue background. The "Create Account" title is in white. Below it, there are two white input fields: the first contains the text "RAVI" and the second contains seven dots, indicating a password. Below the password field is a green "Sign Up" button, and below that is a white "Back to Login" button.

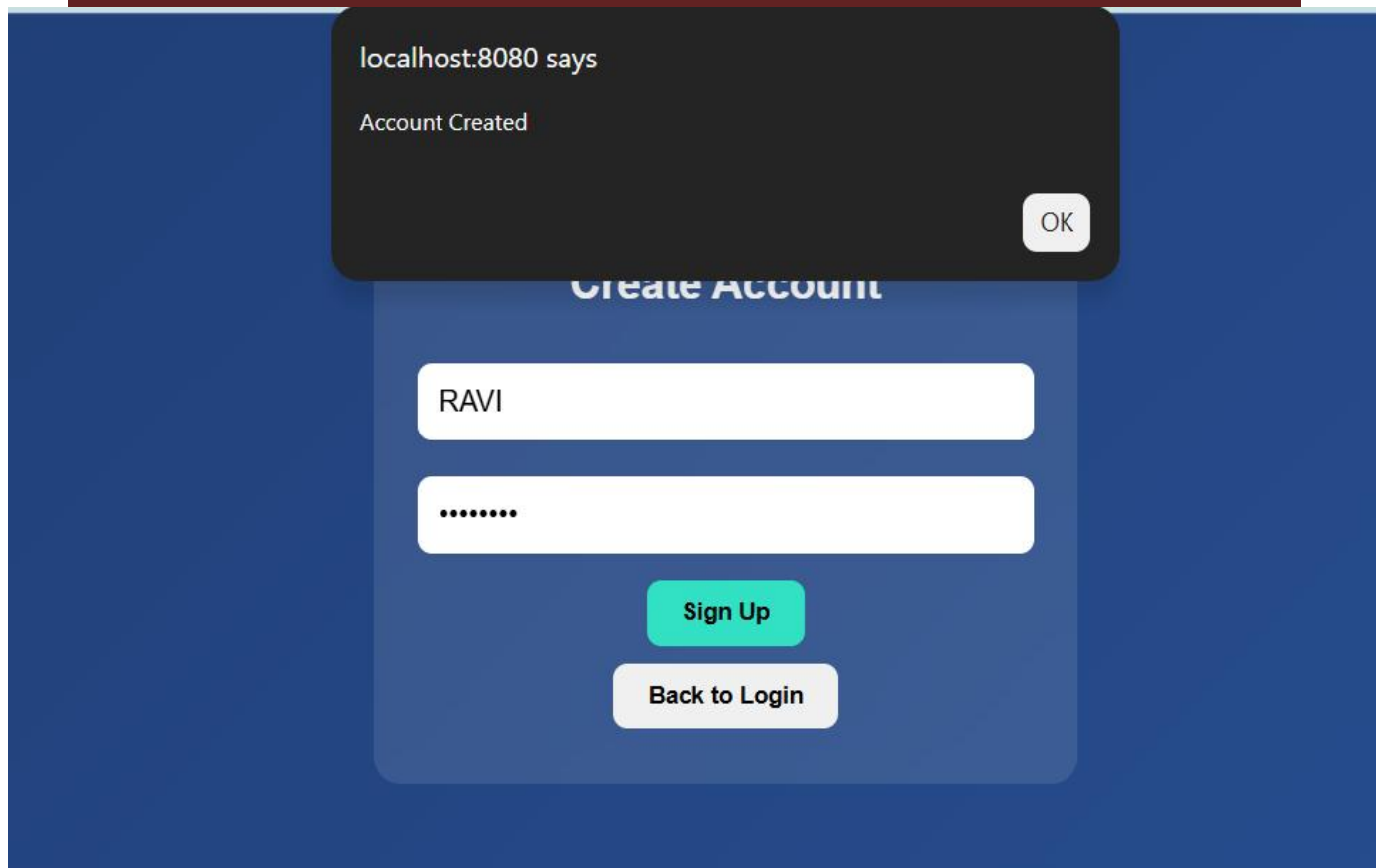
Smart Retail Billing

Create Account

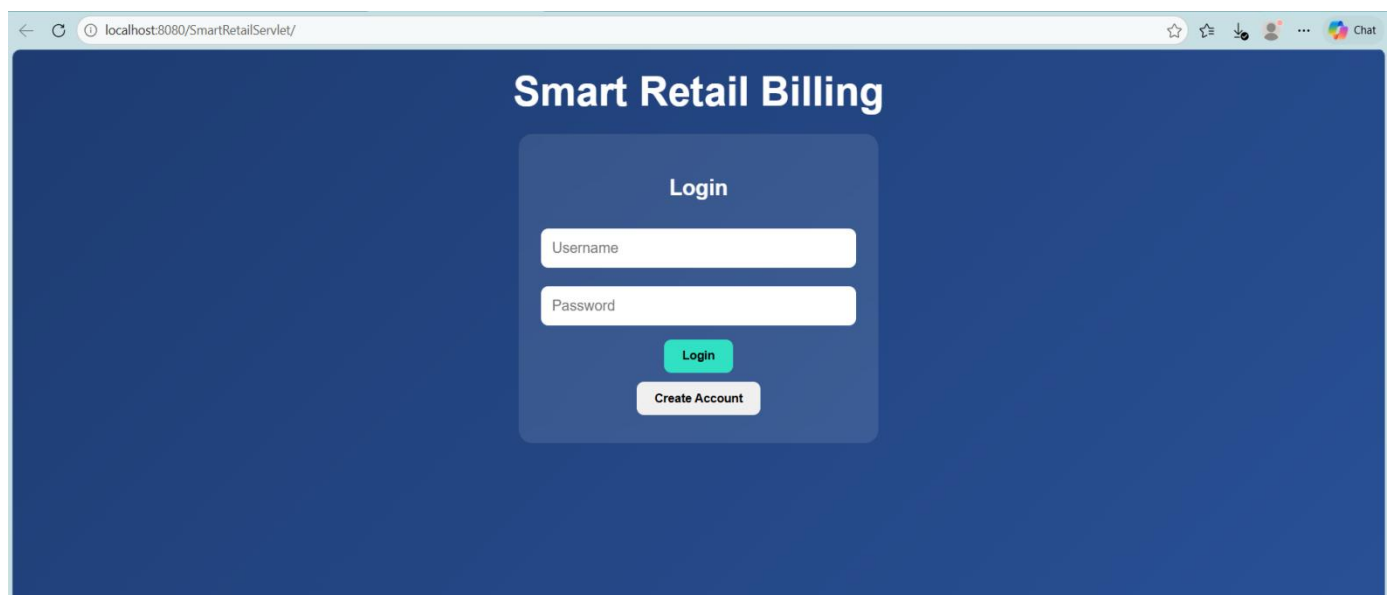
Sign Up

Back to Login

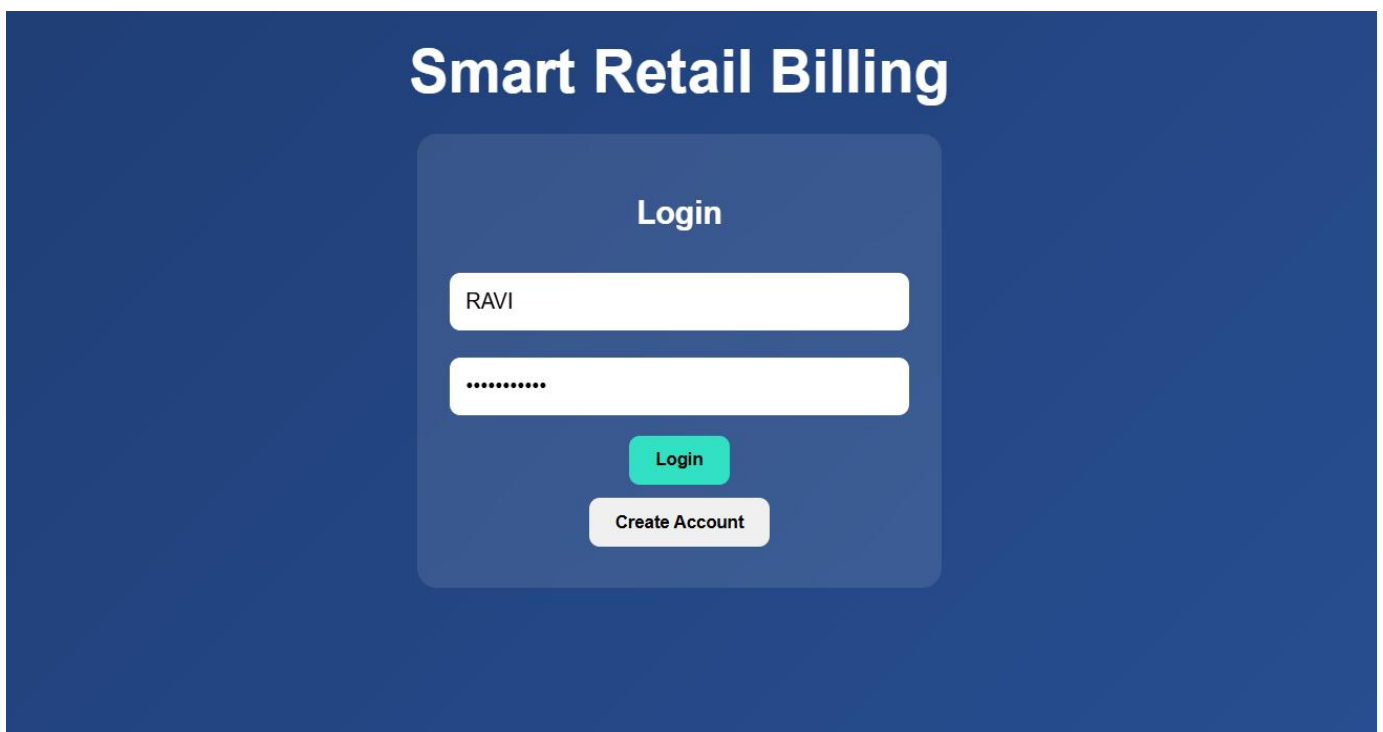
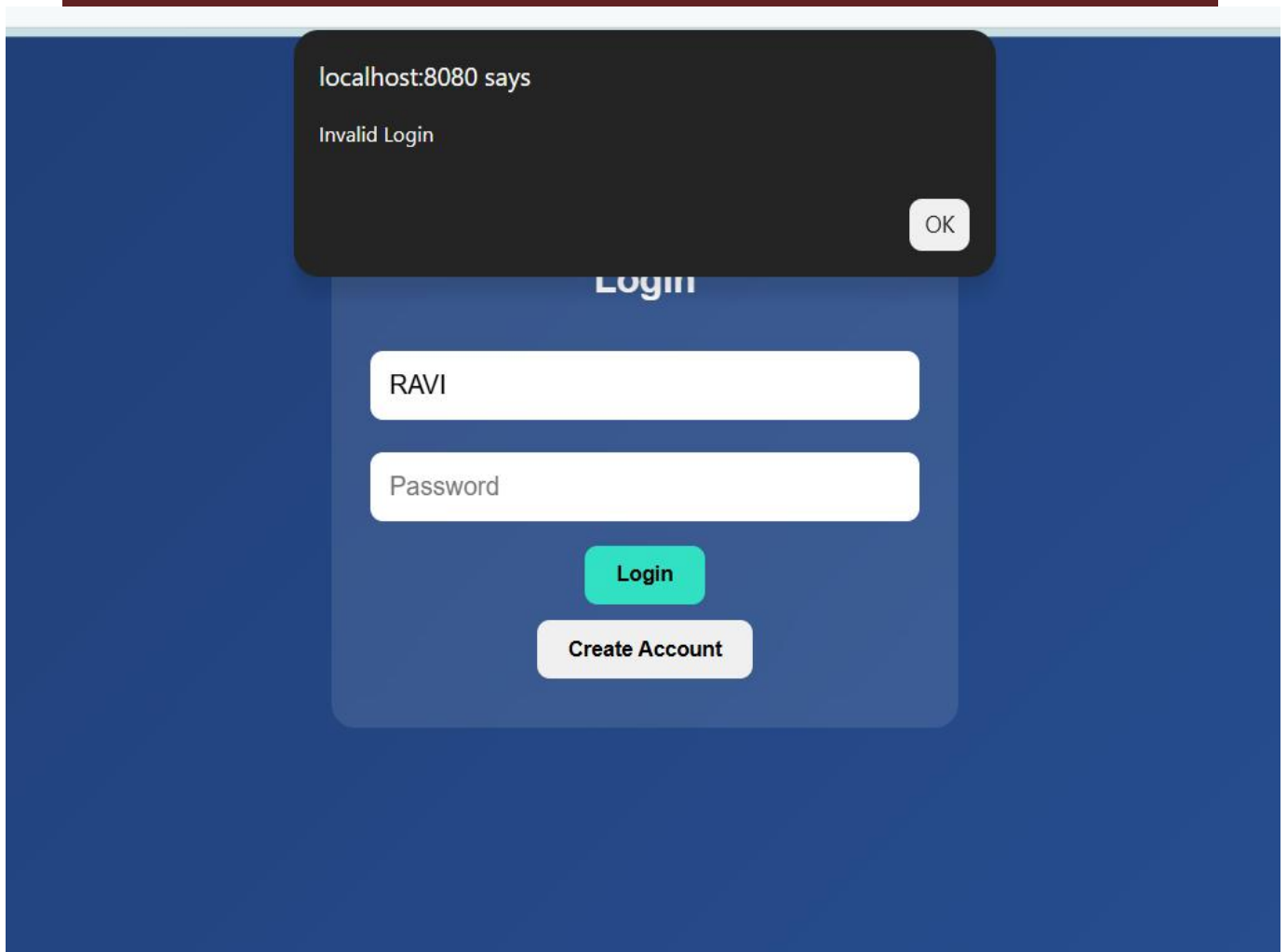
Smart Retail Billing System



A screenshot of a web application interface for creating an account. A dark grey toast notification is displayed at the top, stating "localhost:8080 says" and "Account Created" with an "OK" button. Below the notification is a "Create Account" form. The form has a title "Create Account" and two input fields: the first contains "RAVI" and the second contains masked characters ".....". Below the inputs are two buttons: a green "Sign Up" button and a grey "Back to Login" button.



A screenshot of a web browser showing the "Smart Retail Billing" login page. The browser's address bar displays "localhost:8080/SmartRetailServlet/". The page has a dark blue background with the title "Smart Retail Billing" at the top. In the center is a "Login" form with two input fields labeled "Username" and "Password". Below these fields are two buttons: a green "Login" button and a grey "Create Account" button.



Smart Retail Billing

Billing Dashboard

Logout

Customer Details

RAVI

Add Product

Product Name

Price

Stock

Add Product

Add Product

Product Name

Price

Stock

Add Product

Products

Name	Price	Stock	Qty	Action
------	-------	-------	-----	--------

Cart

Name	Qty	Price	Total	Action
------	-----	-------	-------	--------

Total: ₹0

Generate BillClear Cart

localhost:8080 says

Product Added Successfully

OK

Add Product

Add Product

Products				
Name	Price	Stock	Qty	Action
ICE-CREAM	90	12	1	<button>Add</button>
BISCUIT	45	15	1	<button>Add</button>
MANGO	400	15	1	<button>Add</button>

Cart				
Name	Qty	Price	Total	Action
Total: ₹0				
<button>Generate Bill</button>		<button>Clear Cart</button>		

Smart Retail Billing System

Products

Name	Price	Stock	Qty	Action
ICE-CREAM	90	11	1	<button>Add</button>
BISCUIT	45	15	1	<button>Add</button>
MANGO	400	14	1	<button>Add</button>

Cart

Name	Qty	Price	Total	Action
ICE-CREAM	1	90	90	<button>Delete</button>
MANGO	1	400	400	<button>Delete</button>

Total: ₹490

Generate BillClear Cart

Print

Total: 1 page

Printer

Save as PDF

Layout

☒ Portrait

☐ Landscape

Pages

☒ All

☐ e.g. 1-5, 8, 11-13

More settings

Print using system dialog... (Ctrl+Shift+P)

Save

Cancel

SMART RETAIL BILL

SMART RETAIL BILL

Customer: RAVI

MANGO

Qty: 1

Price: ₹400

Total: ₹400

ICE-CREAM

Qty: 1

Price: ₹90

Total: ₹90

Grand Total: ₹490

CHAPTER 6

OUTPUT (Backend Output)

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| smart_retail |
| smart_retail_db |
| smartretail |
| sys |
+-----+
7 rows in set (0.020 sec)
```

```
mysql> USE smartretail;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_smartretail |
+-----+
| products |
+-----+
1 row in set (0.018 sec)
```

```
mysql> SELECT * FROM products;
+----+-----+-----+-----+
| id | name      | price | quantity |
+----+-----+-----+-----+
| 1  | Book      | 450   | 20        |
| 2  | Book      | 450   | 20        |
| 3  | Phone     | 20000 | -4        |
| 4  | Bottle    | 500   | 0         |
| 5  | Bottle    | 500   | 3         |
| 6  | ICE-CREAM | 90    | 12        |
| 7  | BISCUIT   | 45    | 15        |
| 8  | MANGO     | 400   | 15        |
+----+-----+-----+-----+
8 rows in set (0.012 sec)
```

CHAPTER 6

PROGRAM EXPLANATION – EXPENSE TRACKER SYSTEM

The Smart Retail Billing System is developed using Java web technologies and object-oriented programming principles.

The application works as follows:

User logs into the system using username and password.

Products are added using frontend forms.

Product details are stored in MySQL database.

Products are displayed dynamically in table format.

Products can be added into shopping cart.

Quantity and total price are calculated automatically.

A bill is generated dynamically for the customer.

Stock quantity is reduced automatically after billing.

Features of the System

User Login and Signup

Product Management

Shopping Cart

Dynamic Billing

Automatic Stock Update

Database Connectivity

User Friendly Interface

CHAPTER 7

CONCLUSION

The Smart Retail Billing System has been successfully designed and implemented using Java, Servlets, JDBC, HTML, CSS, JavaScript, and MySQL database connectivity.

The project helps automate retail billing operations and inventory management. It reduces manual work, improves billing speed, and ensures accurate stock maintenance.

The project also provides practical understanding of:

Java Web Development

JDBC Connectivity

Database Management

Object-Oriented Programming Concepts

Thus, the Smart Retail Billing System is an efficient solution for modern retail shop management.

HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements

Processor: Intel i3 or above

RAM: 4 GB or above

Hard Disk: 1 GB free space

Keyboard and Monitor

Software Requirements

Operating System: Windows 10/11

Java JDK 8 or above

Eclipse IDE

Apache Tomcat Server

MySQL Database

JDBC Driver

HTML, CSS, JavaScript

REFERENCES

Herbert Schildt – Java: The Complete Reference

Oracle Java Documentation

Apache Tomcat Documentation

MySQL Official Documentation

GeeksforGeeks Java Tutorials

Eclipse IDE Documentation

